

NumPy and Matplotlib

EEC 3150

Trevecca Nazarene University

Outline

1 Gradient Descent

2 Simulated Annealing

Gradient Descent

Optimization:

Optimization is the process of finding the minimum/maximum value of a function $f(x)$ over some domain D :

$$\min_{x \in D} f(x)$$

$$\max_{x \in D} f(x)$$

- f is called the **objective function**
- D is called the search space
- A value of x which minimizes/maximizes f is called a minimum/maximum (more generally, extremum)

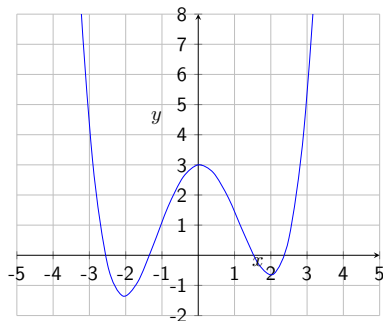
Applications of optimization:

- Engineering: used in computer-aided design (e.g., finding the ideal plane wing to minimize drag but maximize lift)
- Physics: used to fit models of physical systems to observed data (my research!)
- Machine learning: used to train models (e.g., minimizing the loss function in neural networks)
- Operations research: used to solve problems in logistics, resource allocation, scheduling, and other areas of business and industry
- Finance: used to optimize investment portfolios and manage risk
- Game theory: used to model and analyze strategic interactions between agents

Random Walks

Global vs. local:

- A **global minimum** is a value x^* such that $f(x^*) \leq f(x)$ for all $x \in D$
- A **local minimum** is a value x^* such that $f(x^*) \leq f(x)$ for all x in a neighborhood of x^* (meets the calculus criteria for extremum, but isn't an extremal value)



Gradient descent (GD):

- Gradient descent is an optimization method which uses the gradient/derivate of a function to find its minimum
- Given a starting point (x_0), it iteratively generates points by traveling along the direction of the function's steepest descent, i.e., the negative of the gradient
- The learning rate α (usually less than 1) controls the speed of descent

Algorithm Gradient descent

- 1: Set: learning rate α , number of iterations N
 - 2: Set: x_0
 - 3: **for** $i = 0$ to $N - 1$ **do**
 - 4: $x_{i+1} = x_i - \alpha \nabla f(x_i)$
 - 5: **end for**
 - 6: Return: x_N
-

Problems with gradient descent:

Gradient descent is one of the simplest optimization methods and, thus, has many shortcomings:

- It can get stuck in local minima
- It can oscillate b/t solutions
- Gradients can explode

Simulated Annealing

Simulated annealing (SA):

- Simulated annealing is a more robust optimization method that is immune to many of the pitfalls of gradient descent
- It is based on the metalurgical process of annealing, where a metal is heated to a high temperature and then cooled slowly, causing it to become stronger by allowing the molecules in it to find a more optimal, low-energy state
- Given a starting point (x_0), it randomly generates potentially better candidate solutions from a probability distribution (usually a normal centered on the current solution)
- If the candidate solution is better than the current solution, accept it
- If the candidate solution is worse than the current solution, maybe accept it or not (depending on how much worse it is)
- This helps prevent SA from getting stuck in local minima
- As the method runs, it becomes pickier about accepting worse solutions

Some terms:

- Proposal distribution $q(x', x_i)$: the distribution from which new candidate solutions x' are drawn
Example: normal distribution centered on the current solution x_i
Proposal width: the stddev of the normal
- Cooling schedule: the function which defines the rate at which the “temperature” decreases
Example: $T_i = T_0 \exp(-i/\tau)$
- Acceptance probability $\alpha(x', x_i)$: the probability that a candidate solution will be accepted

Simulated Annealing

Algorithm Simulated Annealing

```
1: Set: number of iterations  $N$ , cooling schedule
2: Set:  $x_0$ 
3: for  $i = 0$  to  $N - 1$  do
4:   Generate new candidate
      $x' \sim q(x', x_i)$ 
5:   Calculate change in function value
      $\Delta f = f(x') - f(x_i)$ 
6:   if  $\Delta f < 0$  then
7:     Accept candidate
        $x_{i+1} = x'$ 
8:   else
9:     Accept candidate w/ probability  $\alpha(x', x_i) = \exp(-\Delta f / T_i)$ 
       Either  $x_{i+1} = x'$  or  $x_{i+1} = x_i$ 
10:  end if
11: end for
```

Problems with simulated annealing:

Though SA addresses some of the issues with GD, it is still imperfect:

- It usually converges slowly
- Requires parameter tuning/adaptation
- Can still get stuck in local minima, but not nearly as badly as GD

Though SA addresses some of the issues with GD, it is still imperfect:

- It usually converges slowly
- Requires parameter tuning/adaptation
- Can still get stuck in local minima, but not nearly as badly as GD

Monte Carlo Simulation

Calculate statistics of a complex distribution:

A cannon can fire a cannonball with an initial velocity v_i at an angle θ

$$x_f = x_i + v_i \cos(\theta)t$$

$$y_f = y_i + v_i \sin(\theta)t - \frac{1}{2}gt^2$$

Assuming that the cannon can only launch the ball at $v_i = 100$ m/s, use simulated annealing to determine what angle the cannon must be aimed to hit a target $\hat{x}_f = 500$ m away

Model $(x_i, y_i, y_f = 0)$:

$$x_f(\theta) = \frac{v_i^2 \sin(2\theta)}{g}$$

Objective function (squared error):

$$f(\theta) = \left(\hat{x}_f - x_f(\theta) \right)^2$$